Instructions and Addressing Techniques

Instruction Execution Cycle & Addressing Modes — Direct, Indirect, Relative, Paging, Indexed

Introduction

In computer architecture, machine instructions and addressing techniques determine how a CPU interprets commands and locates the data (operands) required for execution. An instruction represents the fundamental command structure parsed by the control unit, while addressing techniques (or modes) act as the formal rules for calculating the actual physical memory address of the data. Together, they govern program execution efficiency, code compactness, and memory management. The repeated procedure by which the CPU fetches, interprets, and carries out these instructions is known as the **Instruction Execution Cycle**. This document covers that cycle and five key addressing techniques — Direct, Indirect, Relative, Paging, and Indexed.

1. Instruction Execution Cycle

The Instruction Execution Cycle (also called the Fetch–Decode–Execute Cycle) is the sequence a CPU repeatedly follows to run every instruction in a program. It is the core mechanism of the von Neumann stored-program architecture.

1.1 Key Registers

- **Program Counter (PC):** Holds the address of the next instruction to fetch.
- **Memory Address Register (MAR):** Holds the address being accessed in memory.
- **Memory Buffer Register (MBR/MDR):** Temporarily holds data read from or written to memory.
- **Instruction Register (IR):** Holds the currently executing instruction.
- **Accumulator / General-Purpose Registers:** Hold operands and intermediate results.

1.2 Steps of the Instruction Execution Cycle

Step 1 — Fetch

The PC value is copied to the MAR. The CPU fetches the instruction at that address into the MBR, moves it to the IR, then increments the PC to point to the next instruction.

Step 2 — Decode

The Control Unit reads the opcode in the IR to identify the operation (e.g., ADD, LOAD, JUMP) and the addressing mode. It then generates the required internal control signals.

Step 3 — Calculate Operand Address

The Effective Address (EA) of the operand is computed based on the addressing mode — directly from the instruction, via a pointer, by adding an offset to the PC or index register, or through a page table.

Step 4 — Fetch Operand

The operand is retrieved from the EA (memory or register) and loaded into the MBR or ALU input, ready for the operation.

Step 5 — Execute

The ALU or relevant hardware performs the operation — arithmetic, logical, data transfer, or branch. For branch instructions, the PC is updated to the target address.

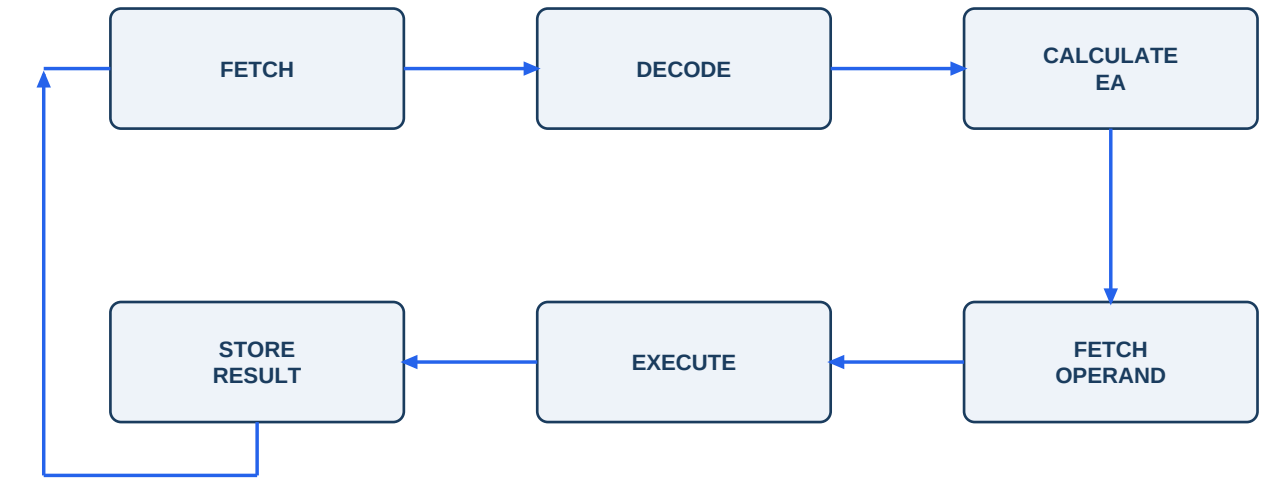
Step 6 — Store Result

If the operation produces a result, it is written back to the designated register or memory location.

Step 7 — Repeat

The cycle immediately restarts at Step 1 using the updated PC and continues until the program ends.

1.3 Cycle Diagram



1.4 Cycle Summary

FETCH → DECODE → CALCULATE EA → FETCH OPERAND → EXECUTE → STORE → REPEAT

This cycle runs at extremely high speed for every instruction, forming the operational heartbeat of the CPU.

2. Addressing Techniques

An addressing mode defines how the CPU interprets the address field of an instruction to locate the actual operand in memory. Each mode offers a different trade-off between speed, flexibility, and memory range. The resulting location is called the Effective Address (EA).

2.1 Direct Addressing

The address field in the instruction is the Effective Address of the operand. The CPU accesses that memory location directly, with no further calculation.

> **EA = A (A is the address field of the instruction)**

Example: *ADD R1, 200* — EA = 200. If Memory[200] = 50 and R1 = 30, the CPU adds 50 to R1, giving R1 = 80, using just one memory access.

Advantages:

- Fast — only one memory reference needed to get the operand.
- Simple to implement and decode.

Limitation:

- Limited address range, restricted by the number of bits in the instruction's address field.

2.2 Indirect Addressing

The address field points to a memory location that holds the actual address of the operand — i.e., the instruction contains a pointer to a pointer.

> **EA = Memory[A] (content at address A is itself the operand's address)**

Example: *ADD R1, @300* — Memory[300] = 500 (pointer), Memory[500] = 50 (operand). The CPU reads 300 → gets 500 → reads 500 → gets 50, then adds it to R1 (two memory accesses).

Advantages:

- Supports a much larger effective address space than what the instruction's address field can hold.
- Ideal for implementing pointers and dynamic data structures.

Limitation:

- Requires two memory references (pointer read + operand read), making it slower than Direct addressing.

2.3 Relative Addressing

The address field holds a signed offset that is added to the current Program Counter value to produce the Effective Address. Primarily used for branch and jump instructions.

> **EA = A + PC (A is a signed displacement relative to the current instruction)**

Example: *JMP +4* at PC = 1000 → EA = 4 + 1000 = 1004. Control jumps to location 1004, 4 locations ahead of the current instruction.

Advantages:

- Produces position-independent, relocatable code — the program can be loaded anywhere in memory.
- The address field only needs to encode a small offset, saving instruction bits.

Limitation:

- Jump range is limited by the size of the offset field.

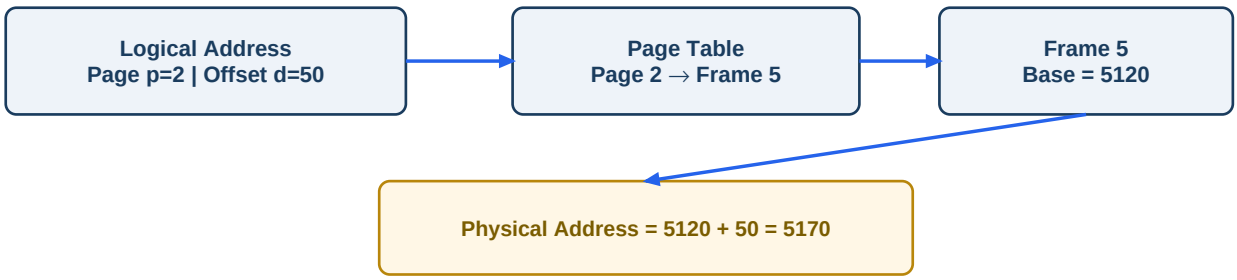
2.4 Paging

Paging divides the logical address space into fixed-size blocks called pages, and physical memory into equal-size blocks called frames. A page table (maintained by the OS/hardware) maps each page number to its corresponding physical frame, enabling virtual-to-physical address translation at runtime.

A logical address is split into: **Page Number (p)** + **Offset (d)**

> **EA = Frame Base Address (from Page Table[p]) + Offset (d)**

Example: Page size = 1024 B, logical address 2098 → page p = 2, offset d = 50. If page 2 maps to frame 5 (base 5120), physical address = 5120 + 50 = 5170.



Advantages:

- Eliminates external fragmentation — any free frame can hold any page.
- Enables virtual memory, allowing programs to use more address space than physical RAM.

Limitations:

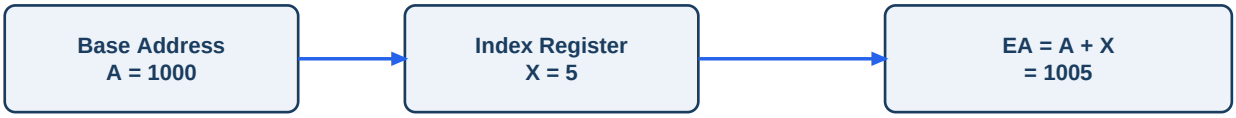
- Page table lookups add overhead; a TLB (Translation Look-aside Buffer) caches recent entries to reduce this.
- Can cause minor internal fragmentation in the last page of a process.

2.5 Indexed Addressing

The Effective Address is obtained by adding the contents of a dedicated Index Register (X) to the address field of the instruction. The index register can be incremented each iteration, making this mode ideal for sequential data access.

> **EA = A + X (X is the content of the Index Register)**

Example: *LOAD R1, 1000(X)*, X = 5 → EA = 1000 + 5 = 1005. Incrementing X each iteration (6, 7, …) steps through successive array elements automatically.



Advantages:

- Best suited for processing arrays, tables, and repetitive sequential data.
- A single instruction can access many locations by simply changing the index register value.

Limitation:

- Requires a dedicated index register and slightly more complex address calculation logic.

2.6 Comparative Summary

Mode	EA Formula	Memory Refs	Best Used For
Direct	EA = A	1	Fixed-location operands
Indirect	EA = Memory[A]	2	Pointers, dynamic structures
Relative	EA = A + PC	1	Branches, relocatable code
Paging	EA = Frame Base[p] + d	1–2 (TLB helps)	Virtual memory management
Indexed	EA = A + X	1	Arrays, tables, loops

Each addressing mode serves a distinct purpose. Direct is the fastest for fixed data; Indirect enables pointers; Relative supports relocatable branching; Paging underpins virtual memory; and Indexed streamlines sequential data access. Together, they give the CPU the flexibility to handle all types of data access patterns efficiently.